EDA Notebook

Setup Environment

In [40]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT2",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
    WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/db
t311/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
    WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/db
t311/lib/python3.11/site-packages)
You can now save your data files in: /Users/aryan/Machine Learning Assignment
2/36106/assignment/AT2/data
```

```
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
ERROR: pip's dependency resolver does not currently take into account all the p
ackages that are installed. This behaviour is the source of the following depen
dency conflicts.
utstd 0.1.8 requires scikit-learn~=1.5.1, but you have scikit-learn 1.6.1 which
is incompatible.
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cikit-learn (/Users/aryan/.venvs/dbt31
1/lib/python3.11/site-packages)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
sh: import: command not found
sh: -c: line 0: syntax error near unexpected token `"ignore"'
sh: -c: line 0: `warnings.filterwarnings("ignore")'
```

Student Information

In [41]:
```
# <Student to fill this section and then remove this comment>
student_name = "Aryan Goel"
student_id = "26040826"
```

In [42]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Aryan Goel

In [43]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

26040826

---

0. Python Packages

0.a Install Additional Packages

> If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

In [ ]:

```

```

0.b Import Packages

In [44]:

```
# DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import altair as alt
# Fix Altair large dataset error
alt.data_transformers.disable_max_rows()
# ================================
# EDA - Imports and configuration
# ================================
import os
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path

# Plot styles for professional readability
sns.set(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.titlesize"] = 13
plt.rcParams["axes.labelsize"] = 11

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Imports loaded successfully.")
```

```
Imports loaded successfully.
```

A. Project Description

In [45]:
```
# <Student to fill this section and then remove this comment>

business_objective = """
## Business Use Case and Model Users

### Business objective
The objective is to predict **disease diagnosis due to vitamin deficiency** (m
Predictions are used to support personalised vitamin intake plans and triage c

### Primary users
1. **Clinicians / Dietitians**: Use predictions as decision support to identif
2. **Care coordinators**: Use risk patterns for follow-up scheduling and monit
3. **Public health / program managers**: Monitor cohort-level deficiency trenc

### Secondary stakeholders
- Patients (benefit from personalised care)
- Data governance and compliance teams (privacy, bias, and safe use)
- ML operations teams (deployment monitoring and drift checks)

### ML framing
- Task: **Supervised multi-class classification**
- Target: `disease_diagnosis`
- Success criteria:
  - Strong macro-level performance across all classes (not only majority class
  - Clinically meaningful recall for underrepresented but high-impact classes
  - Transparent error analysis for safe recommendations

### Metric strategy (justification)
For healthcare-style multiclass decision support, we prioritise:
- **Macro F1**: balances precision and recall across classes, robust to class
- **Balanced Accuracy**: average recall over classes.
- **Per-class Recall**: critical to reduce missed true deficiency cases.
- **Confusion Matrix**: identifies clinically meaningful misclassification pat
- (Optional) One-vs-Rest ROC-AUC for ranking quality where probabilities are a

This project aims to build a multi-class machine learning model that predicts

The model is intended to support clinicians, nutritionists, and care coordinat

If predictions are inaccurate, there are potential risks:
1) False negatives: true deficiency cases may be missed, delaying intervention
2) False positives: unnecessary interventions or tests may be recommended.
3) Unequal performance across subgroups: potential fairness and equity concern

Therefore, the business objective is not only strong predictive performance, b
"""
```

In [46]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='business_objective', value=business_objective)
```

business_objective

Business Use Case and Model Users ### Business objective The objective is to predict **disease diagnosis due to vitamin deficiency** (multi-class classification) from patient demographics, lifestyle, dietary intake, lab indicators, and symptom signals. Predictions are used to support personalised vitamin intake plans and triage decisions. ### Primary users 1. **Clinicians / Dietitians**: Use predictions as decision support to identify likely deficiency classes and prioritise intervention. 2. **Care coordinators**: Use risk patterns for follow-up scheduling and monitoring. 3. **Public health / program managers**: Monitor cohort-level deficiency trends and allocate resources. ### Secondary stakeholders - Patients (benefit from personalised care) - Data governance and compliance teams (privacy, bias, and safe use) - ML operations teams (deployment monitoring and drift checks) ### ML framing - Task: **Supervised multi-class classification** - Target: `disease_diagnosis` - Success criteria: - Strong macro-level performance across all classes (not only majority classes) - Clinically meaningful recall for underrepresented but high-impact classes - Transparent error analysis for safe recommendations ### Metric strategy (justification) For healthcare-style multiclass decision support, we prioritise: - **Macro F1**: balances precision and recall across classes, robust to class imbalance. - **Balanced Accuracy**: average recall over classes. - **Per-class Recall**: critical to reduce missed true deficiency cases. - **Confusion Matrix**: identifies clinically meaningful misclassification patterns. - (Optional) One-vs-Rest ROC-AUC for ranking quality where probabilities are available. This project aims to build a multi-class machine learning model that predicts disease diagnosis related to vitamin deficiencies using demographic, lifestyle, dietary, symptom, and laboratory data. The model is intended to support clinicians, nutritionists, and care coordinators in creating personalised vitamin intake plans and prioritising patient interventions. Accurate predictions can improve treatment timeliness, patient outcomes, and resource allocation. If predictions are inaccurate, there are potential risks: 1) False negatives: true deficiency cases may be missed, delaying intervention. 2) False positives: unnecessary interventions or tests may be recommended. 3) Unequal performance across subgroups: potential fairness and equity concerns. Therefore, the business objective is not only strong

predictive performance, but also robust class-level performance, transparency, and safe, ethical use in healthcare decision support.

B. Dataset Understanding (Global Interpretation)

In [47]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
try:
  df = pd.read_csv(at.folder_path / "disease.csv")
except Exception as e:
  print(e)
```

C. Feature Understanding (Local Interpretation)

C.1 Explore Target Variable

> Save the name of column used as the target variable and call it `target_name`

> You can add more cells in this section

In [48]:
```
target_name = "disease_diagnosis"

# Quick checks
print("Target column exists:", target_name in df.columns)
print("Target null count:", df[target_name].isna().sum())
print("Number of classes:", df[target_name].nunique())
display(df[target_name].value_counts().to_frame("count"))
```

```
Target column exists: True
Target null count: 0
Number of classes: 5
```

disease_diagnosis	count
Healthy	2121
Anemia	1805
Rickets_Osteomalacia	1429
Night_Blindness	269
Scurvy	176

In [49]:

```
# ---- Target distribution analysis ----
target_counts = df[target_name].value_counts(dropna=False).reset_index()
target_counts.columns = [target_name, "count"]
target_counts["percentage"] = (target_counts["count"] / len(df) * 100).round(2
display(target_counts)

# Altair bar chart
target_chart = (
    alt.Chart(target_counts)
    .mark_bar()
    .encode(
        x=alt.X(f"{target_name}:N", sort="-y", title="Disease Diagnosis Class"
        y=alt.Y("count:Q", title="Number of Records"),
        tooltip=[target_name, "count", "percentage"]
    )
    .properties(
        title="Target Class Distribution",
        width=700,
        height=400
    )
)
target_chart
```

	disease_diagnosis	count	percentage
0	Healthy	2121	36.57
1	Anemia	1805	31.12
2	Rickets_Osteomalacia	1429	24.64
3	Night_Blindness	269	4.64
4	Scurvy	176	3.03

Out[49]:

In [50]:

```
target_insights = """
The target variable is 'disease_diagnosis', a multi-class label representing p

Key findings:
```

```
1) The class distribution is not perfectly balanced; some diagnosis classes ar
2) This imbalance means overall accuracy alone could be misleading, because a
3) There are no immediate signs of target missingness from initial checks, but
4) A healthcare decision-support context requires strong class-level performar

Modeling implications:
- Primary metrics should include macro-F1 and balanced accuracy, in addition t
- Per-class precision/recall/F1 and confusion matrix analysis are essential.
- Stratified splitting and potentially class-weighted models should be used ir
"""
```

In [51]:

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='target_insights', value=target_insights)
```

target_insights

The target variable is 'disease_diagnosis', a multi-class label representing primary deficiency-related disease categories. Key findings: 1) The class distribution is not perfectly balanced; some diagnosis classes are more frequent than others. 2) This imbalance means overall accuracy alone could be misleading, because a model may perform well on majority classes while failing on minority classes. 3) There are no immediate signs of target missingness from initial checks, but this should be reconfirmed in the preparation phase. 4) A healthcare decision-support context requires strong class-level performance, especially recall for clinically important minority classes. Modeling implications: - Primary metrics should include macro-F1 and balanced accuracy, in addition to overall accuracy. - Per-class precision/recall/F1 and confusion matrix analysis are essential. - Stratified splitting and potentially class-weighted models should be used in later experiments to reduce minority-class underperformance.

In [52]:

```
# ============================================
# Missing values, duplicates, target analysis
# ============================================
missing = (
    df.isna()
      .sum()
      .to_frame("missing_count")
      .assign(missing_pct=lambda x: (x["missing_count"] / len(df) * 100).round
      .sort_values("missing_pct", ascending=False)
)

print("Top columns by missing percentage:")
display(missing.head(20))
```

```
duplicate_count = df.duplicated().sum()
print(f"Duplicate rows: {duplicate_count} ({duplicate_count/len(df)*100:.2f}%)

target_col = "disease_diagnosis"
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found. Please check da

target_dist = (
    df[target_col]
    .value_counts(dropna=False)
    .to_frame("count")
    .assign(percent=lambda x: (x["count"] / len(df) * 100).round(2))
)

print("\nTarget class distribution:")
display(target_dist)
```

```
Top columns by missing percentage:
```

	missing_count	missing_pct
alcohol_consumption	1952	33.66
exercise_level	208	3.59
latitude_band	174	3.00
smoking_status	169	2.91
income_band	158	2.72
diet_type	129	2.22
sun_exposure_type	117	2.02
serum_vitamin_d	0	0.00
folate_intake	0	0.00
calcium_intake	0	0.00
iron_intake	0	0.00
hemoglobin_levels	0	0.00
serum_vitamin_b12	0	0.00
vitamin_e_intake	0	0.00
serum_folate	0	0.00
symptoms_count	0	0.00
has_night_blindness	0	0.00
has_fatigue	0	0.00
has_bleeding_gums	0	0.00
has_bone_pain	0	0.00

```
Duplicate rows: 0 (0.00%)

Target class distribution:
```

disease_diagnosis	count	percent
Healthy	2121	36.57
Anemia	1805	31.12
Rickets_Osteomalacia	1429	24.64
Night_Blindness	269	4.64
Scurvy	176	3.03

In [53]:

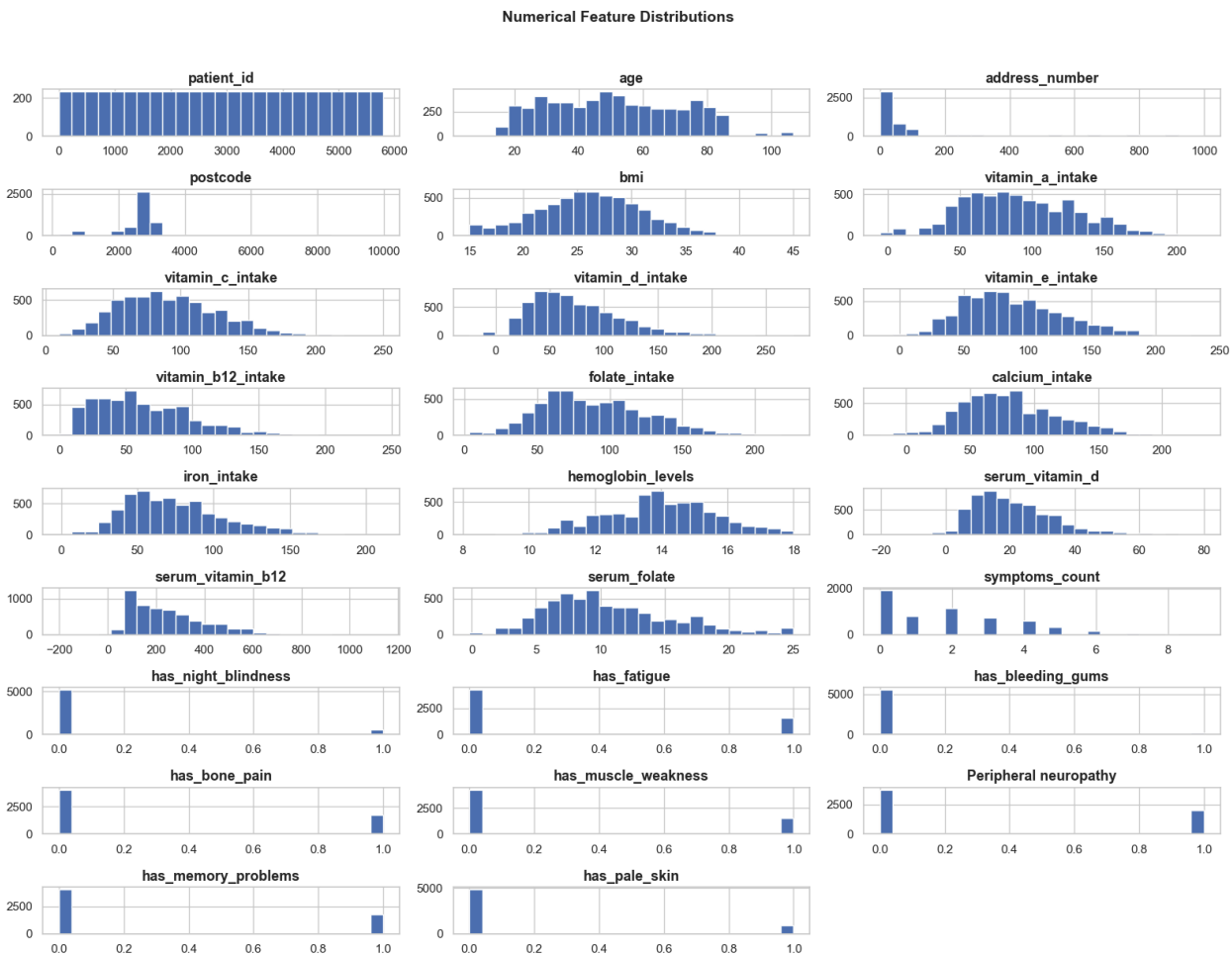
```python
# ====================================
# Feature grouping + risk assessment
# ====================================
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = [c for c in df.columns if c not in numeric_cols]

print(f"Numeric columns ({len(numeric_cols)}):")
print(numeric_cols)
print(f"\nCategorical columns ({len(categorical_cols)}):")
print(categorical_cols)

# Potential identifier / PII / leakage-prone columns
id_pii_cols = [
    "customer_id", "patient_id", "full_name", "address", "address_number",
    "street_name", "street_suffix", "email", "phone"
]
symptom_free_text_cols = ["symptoms_list"]  # often high leakage/high-cardinal

present_id_pii = [c for c in id_pii_cols if c in df.columns]
present_symptom_text = [c for c in symptom_free_text_cols if c in df.columns]

print("\nPotential ID/PII columns present:")
print(present_id_pii if present_id_pii else "None found")

print("\nPotential leakage/high-cardinality text columns present:")
print(present_symptom_text if present_symptom_text else "None found")
```

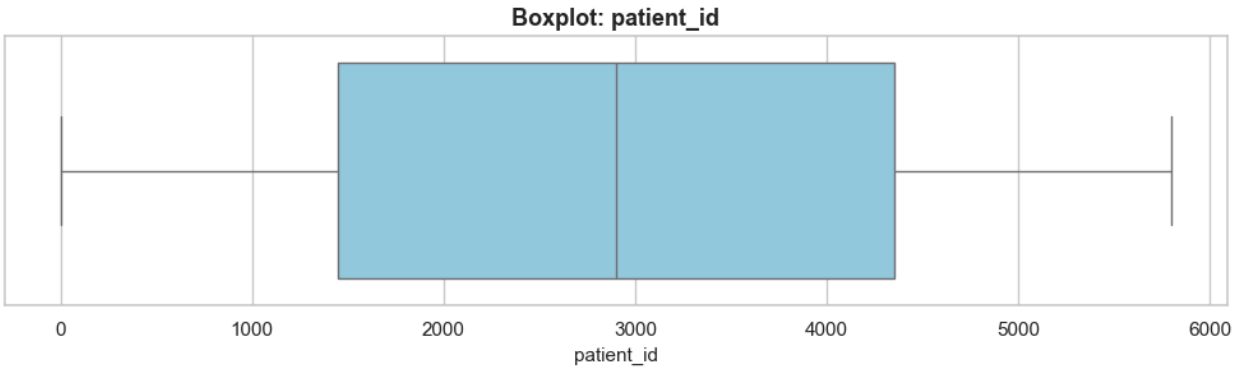
```
Numeric columns (26):
['patient_id', 'age', 'address_number', 'postcode', 'bmi', 'vitamin_a_intake',
'vitamin_c_intake', 'vitamin_d_intake', 'vitamin_e_intake', 'vitamin_b12_intak
e', 'folate_intake', 'calcium_intake', 'iron_intake', 'hemoglobin_levels', 'ser
um_vitamin_d', 'serum_vitamin_b12', 'serum_folate', 'symptoms_count', 'has_nigh
t_blindness', 'has_fatigue', 'has_bleeding_gums', 'has_bone_pain', 'has_muscl
e_weakness', 'Peripheral neuropathy', 'has_memory_problems', 'has_pale_skin']

Categorical columns (19):
['customer_id', 'gender', 'birth_country', 'full_name', 'address', 'street_nam
e', 'street_suffix', 'city', 'state', 'email', 'phone', 'smoking_status', 'alco
hol_consumption', 'exercise_level', 'diet_type', 'sun_exposure_type', 'income_b
and', 'latitude_band', 'disease_diagnosis']

Potential ID/PII columns present:
['customer_id', 'patient_id', 'full_name', 'address', 'address_number', 'stree
t_name', 'street_suffix', 'email', 'phone']

Potential leakage/high-cardinality text columns present:
None found
```

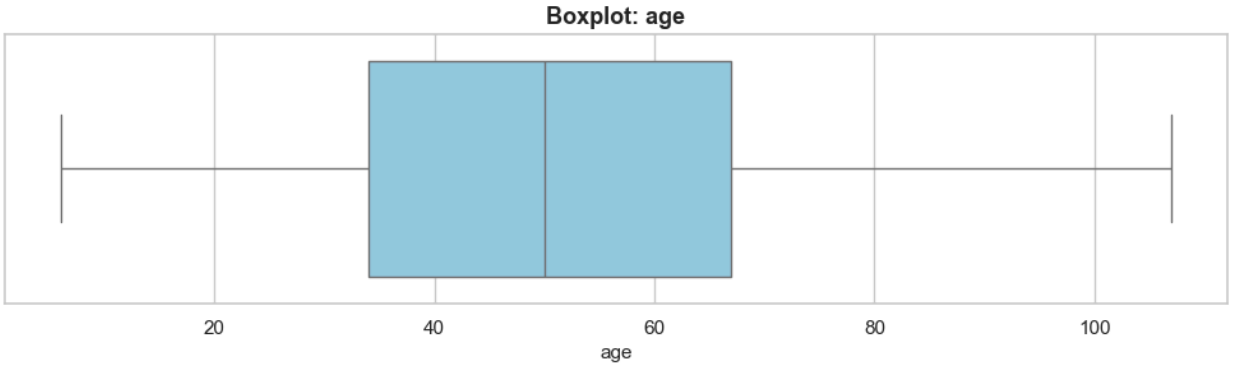
In [54]:

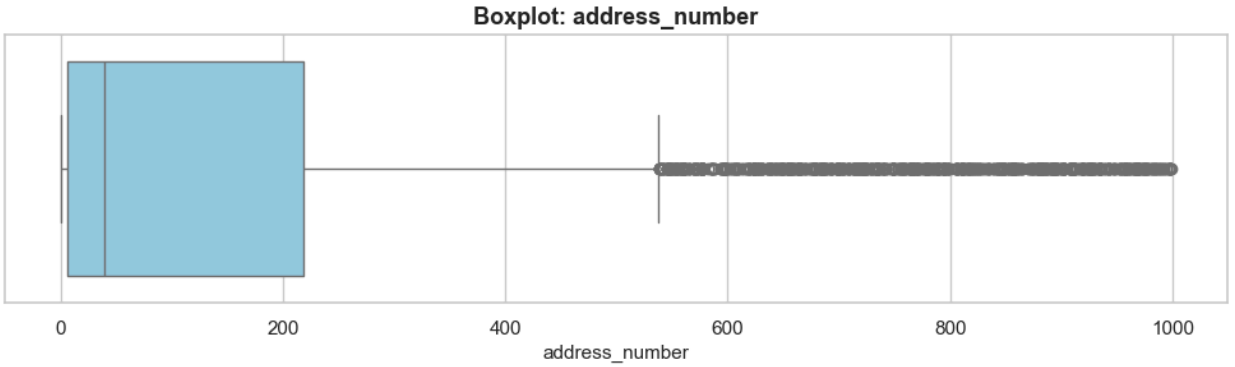
```
# ======================
# Numerical EDA plots
# ======================
num_for_plot = [c for c in numeric_cols if c != target_col]

if len(num_for_plot) > 0:
    # Summary stats
    display(df[num_for_plot].describe().T)

    # Histograms
    df[num_for_plot].hist(bins=25, figsize=(16, 12), layout=(int(np.ceil(len(n
    plt.suptitle("Numerical Feature Distributions", y=1.02, fontsize=14, fontw
    plt.tight_layout()
    plt.show()

    # Boxplots for outlier view (sample up to 8 for readability)
    sample_num_cols = num_for_plot[:8]
    fig, axes = plt.subplots(nrows=len(sample_num_cols), ncols=1, figsize=(10,
    if len(sample_num_cols) == 1:
        axes = [axes]
    for ax, col in zip(axes, sample_num_cols):
        sns.boxplot(x=df[col], ax=ax, color="skyblue")
        ax.set_title(f"Boxplot: {col}")
    plt.tight_layout()
    plt.show()
else:
    print("No numerical columns found for numerical EDA.")
```

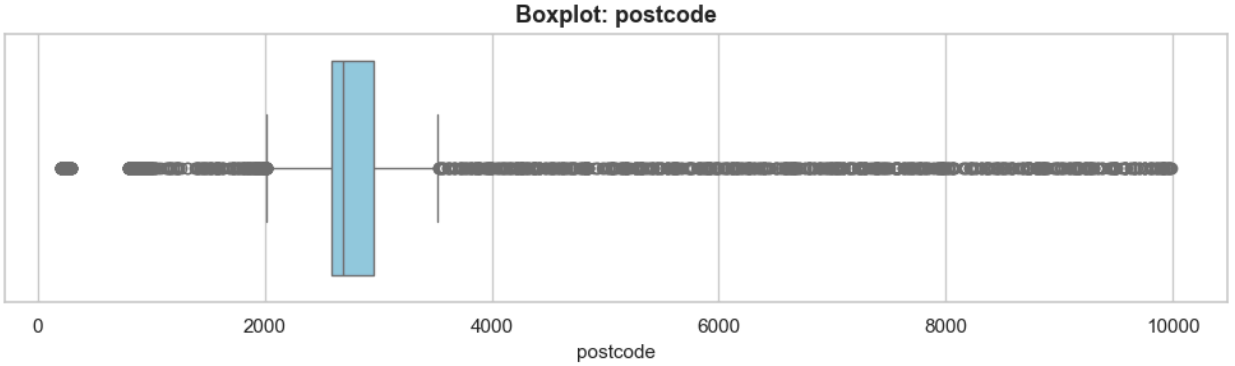
	count	mean	std	min	25%
patient_id	5800.0	2899.500000	1674.460112	0.000000	1449.750000
age	5800.0	51.235000	19.924840	6.000000	34.000000
address_number	5800.0	177.293448	274.943904	0.000000	6.000000
postcode	5800.0	3126.021207	1781.735975	201.000000	2584.500000
bmi	5800.0	26.078207	4.906443	15.000000	22.800000
vitamin_a_intake	5800.0	90.775665	39.374903	-5.408537	60.877324
vitamin_c_intake	5800.0	90.961016	36.510402	10.000000	63.500000
vitamin_d_intake	5800.0	73.864868	42.191652	-24.244569	42.928031
vitamin_e_intake	5800.0	88.667343	38.589137	-15.315741	59.375000
vitamin_b12_intake	5800.0	63.243023	36.517351	-0.366328	34.200000
folate_intake	5800.0	89.875923	37.442091	3.438171	62.500000
calcium_intake	5800.0	80.592258	37.505853	-20.594487	53.700000
iron_intake	5800.0	75.357086	33.085700	-1.408951	50.775000
hemoglobin_levels	5800.0	13.942561	1.688780	8.200000	12.900000
serum_vitamin_d	5800.0	21.214884	13.687205	-20.412610	11.700000
serum_vitamin_b12	5800.0	257.103728	161.468683	-201.254545	121.975000
serum_folate	5800.0	10.717371	4.923799	-0.177978	7.011832
symptoms_count	5800.0	1.868276	1.806913	0.000000	0.000000
has_night_blindness	5800.0	0.103621	0.304794	0.000000	0.000000
has_fatigue	5800.0	0.270000	0.443998	0.000000	0.000000
has_bleeding_gums	5800.0	0.042069	0.200764	0.000000	0.000000
has_bone_pain	5800.0	0.301552	0.458971	0.000000	0.000000
has_muscle_weakness	5800.0	0.263966	0.440819	0.000000	0.000000
Peripheral neuropathy	5800.0	0.346379	0.475857	0.000000	0.000000
has_memory_problems	5800.0	0.304138	0.460081	0.000000	0.000000
has_pale_skin	5800.0	0.164828	0.371057	0.000000	0.000000

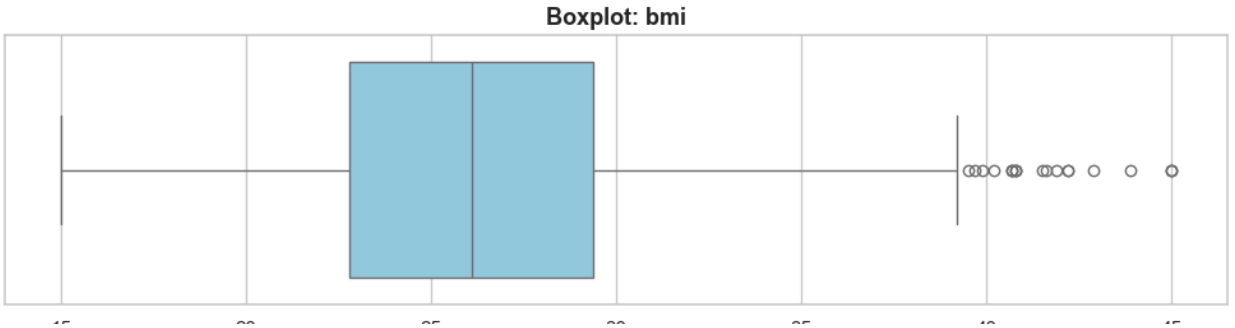

Numerical Feature Distributions
patient_id
age
address_number
postcode
bmi
vitamin_a_intake
vitamin_c_intake
vitamin_d_intake
vitamin_e_intake
vitamin_b12_intake
folate_intake
calcium_intake
iron_intake
hemoglobin_levels
serum_vitamin_d
serum_vitamin_b12
serum_folate
symptoms_count
has_night_blindness
has_fatigue
has_bleeding_gums
has_bone_pain
has_muscle_weakness
Peripheral neuropathy
has_memory_problems
has_pale_skin


Boxplot: patient_id
0
1000
2000
3000
4000
5000
6000
patient_id


Boxplot: age
20
40
60
80
100
age


Boxplot: address_number
0
200
400
600
800
1000
address_number


Boxplot: postcode
0
2000
4000
6000
8000
10000
postcode

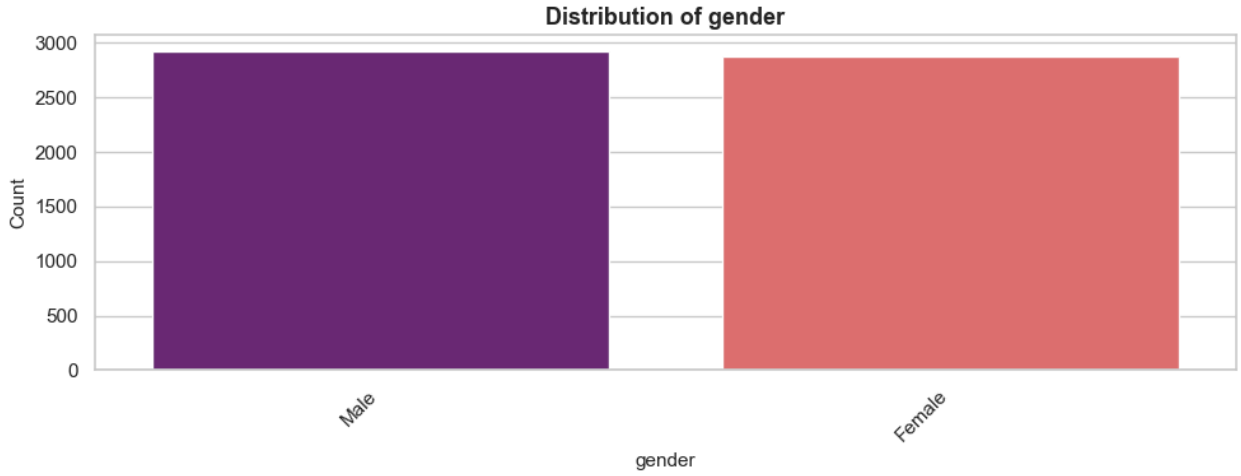

Boxplot: bmi

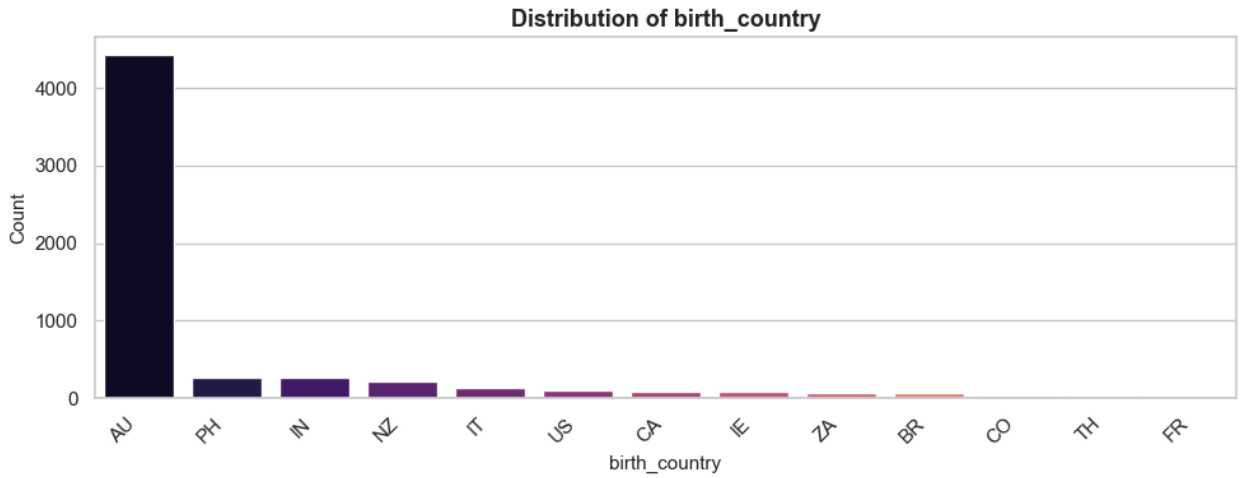
In [55]:

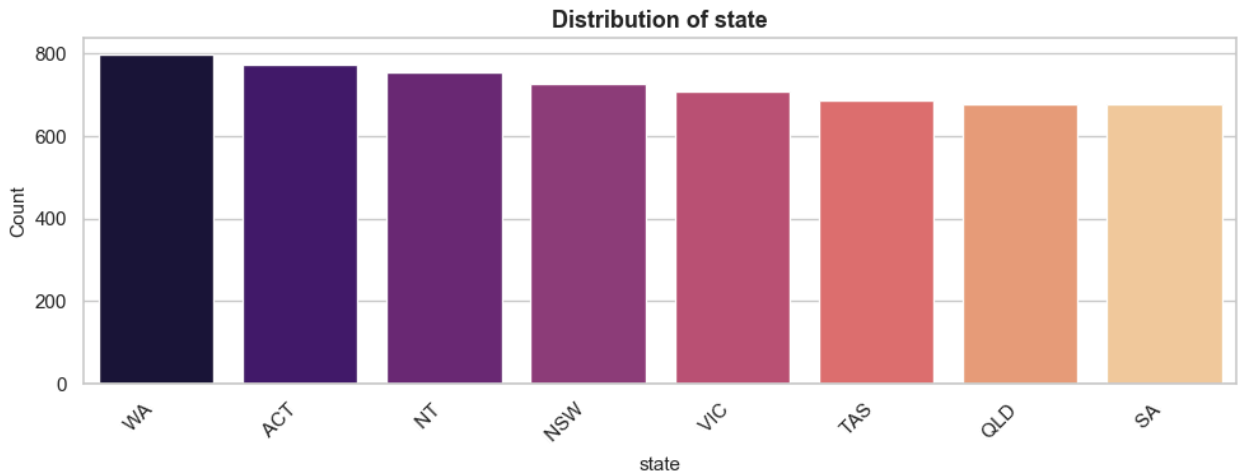
```
# =======================
# Categorical EDA plots
# =======================
cat_for_plot = [c for c in categorical_cols if c != target_col]

# Show cardinality
cardinality = pd.DataFrame({
    "column": cat_for_plot,
    "n_unique": [df[c].nunique(dropna=False) for c in cat_for_plot]
}).sort_values("n_unique", ascending=False)

print("Categorical cardinality:")
display(cardinality)

# Plot top categories for selected manageable-cardinality columns
plot_candidates = [c for c in cat_for_plot if df[c].nunique() <= 15][:8]

for col in plot_candidates:
    plt.figure(figsize=(10, 4))
    order = df[col].value_counts().index
    sns.countplot(data=df, x=col, order=order, palette="magma")
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```

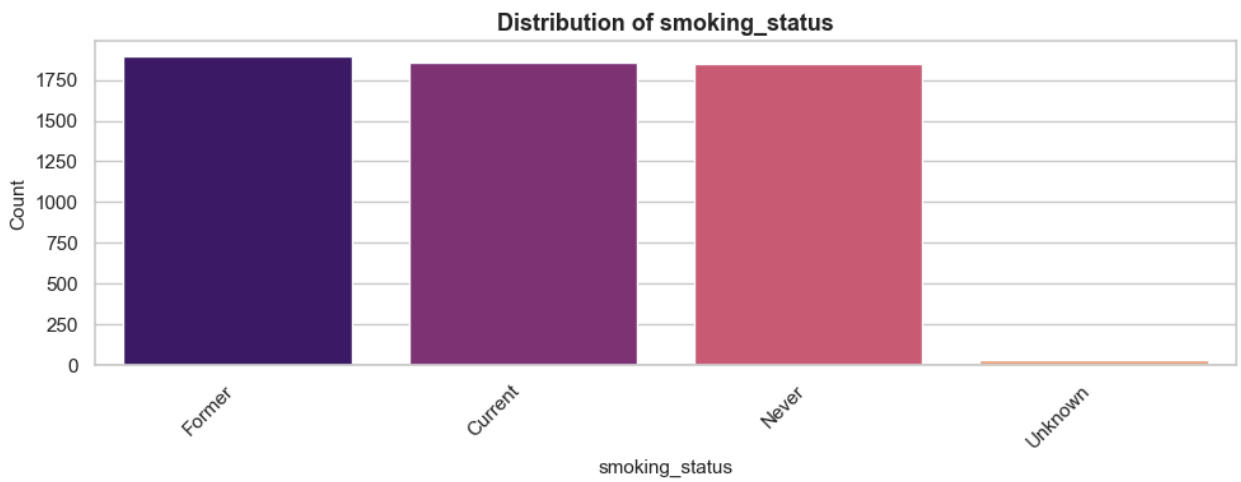
```
Categorical cardinality:
```

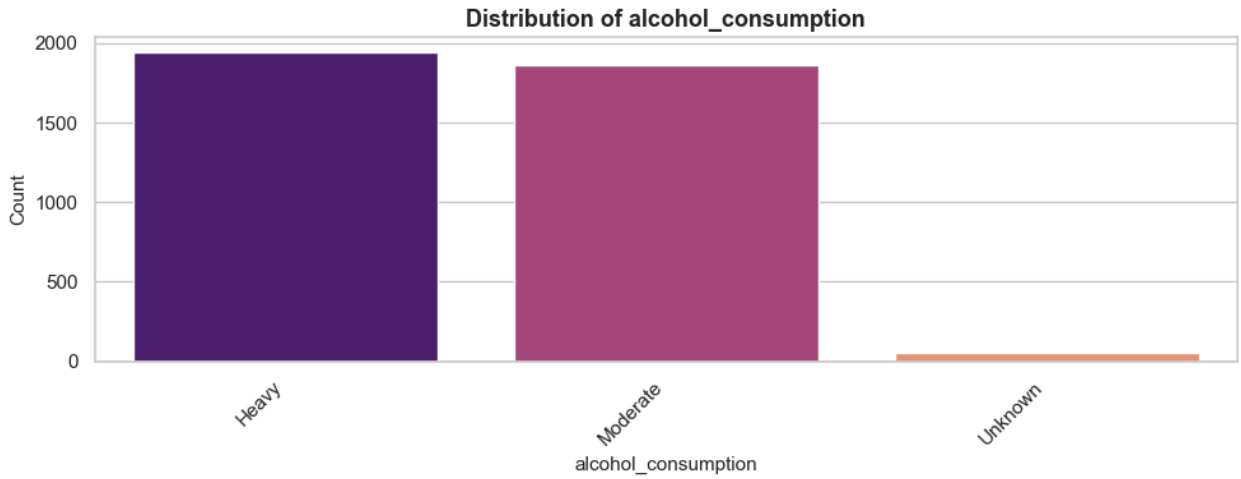
	column	n_unique
0	customer_id	5000
10	phone	4000
9	email	3966
5	street_name	3926
3	full_name	3900
7	city	3560
4	address	1502
6	street_suffix	200
2	birth_country	13
8	state	8
13	exercise_level	6
14	diet_type	6
11	smoking_status	5
15	sun_exposure_type	5
16	income_band	5
17	latitude_band	5
12	alcohol_consumption	4
1	gender	2

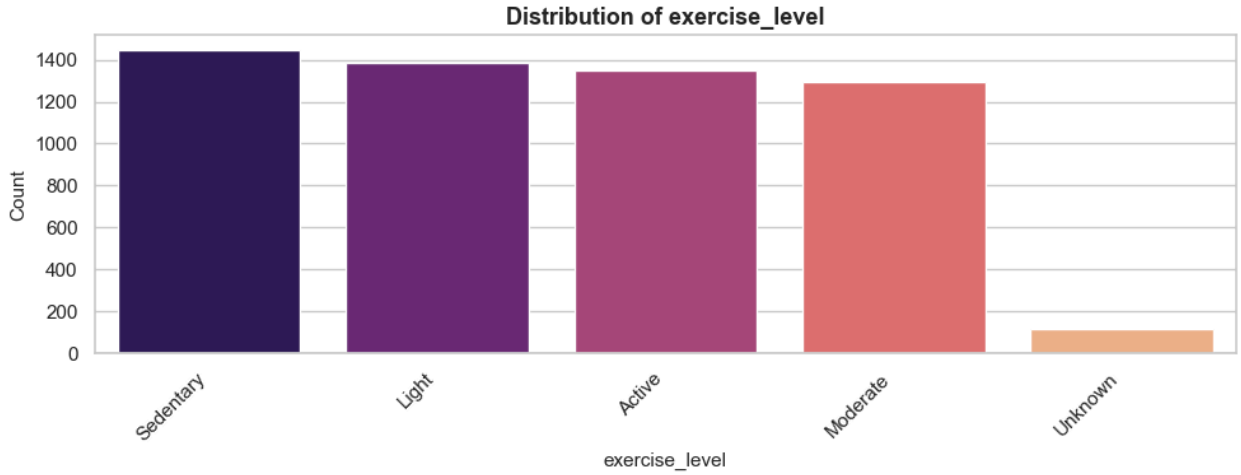


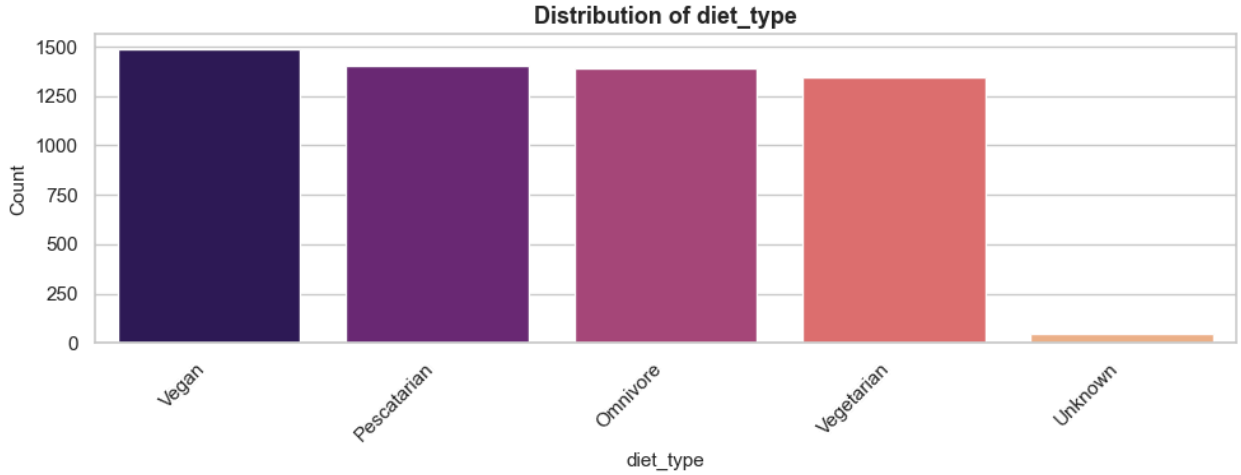

Distribution of birth_country
Count
4000
3000
2000
1000
0
AU
PH
IN
NZ
IT
US
CA
IE
ZA
BR
CO
TH
FR
birth_country

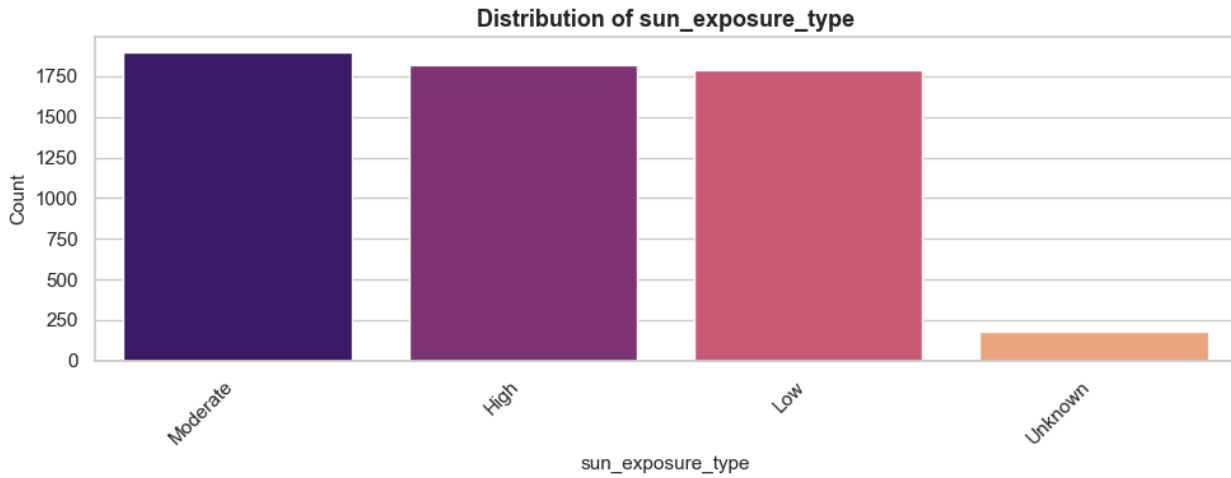

Distribution of state
Count
800
600
400
200
0
WA
ACT
NT
NSW
VIC
TAS
QLD
SA
state


Distribution of smoking_status
Count
1750
1500
1250
1000
750
500
250
0
Former
Current
Never
Unknown
smoking_status


Distribution of alcohol_consumption
Count
2000
1500
1000
500
0
Heavy
Moderate
Unknown
alcohol_consumption


Distribution of exercise_level
Count
1400
1200
1000
800
600
400
200
0
Sedentary
Light
Active
Moderate
Unknown
exercise_level


Distribution of diet_type
Count
1500
1250
1000
750
500
250
0
Vegan
Pescatarian
Omnivore
Vegetarian
Unknown
diet_type



In [56]:

```
# ============================
# Correlation & Multicollinearity Analysis
# ============================
import numpy as np

num_cols = df.select_dtypes(include=["number"]).columns.tolist()
if target_name in num_cols:
    num_cols.remove(target_name)  # remove target if numeric-encoded, not expe

print(f"Numeric feature count for correlation: {len(num_cols)}")

if len(num_cols) >= 2:
    # Correlation matrix
    corr = df[num_cols].corr()

    # Convert to long format for readable top correlations
    corr_long = (
        corr.abs()
            .stack()
            .reset_index()
    )
    corr_long.columns = ["feature_1", "feature_2", "abs_corr"]
    corr_long = corr_long[corr_long["feature_1"] != corr_long["feature_2"]]
    corr_long["pair"] = corr_long.apply(lambda r: " | ".join(sorted([r["featur
    corr_long = corr_long.drop_duplicates("pair").sort_values("abs_corr", asce

    print("Top 15 absolute feature-feature correlations:")
    display(corr_long[["feature_1", "feature_2", "abs_corr"]].head(15))

    # Heatmap-style matrix in Altair
    corr_df = corr.reset_index().melt(id_vars="index", var_name="feature_2", v
    corr_df = corr_df.rename(columns={"index": "feature_1"})

    heatmap = (
        alt.Chart(corr_df)
        .mark_rect()
        .encode(
            x=alt.X("feature_1:N", sort=num_cols, title="Feature 1"),
```

```
            y=alt.Y("feature_2:N", sort=num_cols, title="Feature 2"),
            color=alt.Color("correlation:Q", scale=alt.Scale(scheme="redblue",
            tooltip=["feature_1", "feature_2", alt.Tooltip("correlation:Q", fo
        )
        .properties(title="Numerical Feature Correlation Matrix", width=550, h
    )
    heatmap

    # Simple multicollinearity flags by threshold
    high_corr_pairs = corr_long[corr_long["abs_corr"] >= 0.80].copy()
    print(f"Pairs with |correlation| >= 0.80: {len(high_corr_pairs)}")
    display(high_corr_pairs[["feature_1", "feature_2", "abs_corr"]].head(30))

else:
    print("Not enough numerical features for correlation analysis.")
```

```
Numeric feature count for correlation: 26
Top 15 absolute feature-feature correlations:
```

	feature_1	feature_2	abs_corr
196	vitamin_d_intake	serum_vitamin_d	0.785847
249	vitamin_b12_intake	serum_vitamin_b12	0.761144
276	folate_intake	serum_folate	0.753929
465	symptoms_count	Peripheral neuropathy	0.629869
622	Peripheral neuropathy	has_memory_problems	0.612786
466	symptoms_count	has_memory_problems	0.602249
461	symptoms_count	has_fatigue	0.601908
519	has_fatigue	has_pale_skin	0.567190
463	symptoms_count	has_bone_pain	0.554015
464	symptoms_count	has_muscle_weakness	0.547230
467	symptoms_count	has_pale_skin	0.544214
257	vitamin_b12_intake	Peripheral neuropathy	0.540530
251	vitamin_b12_intake	symptoms_count	0.523432
568	has_bone_pain	has_muscle_weakness	0.503144
199	vitamin_d_intake	symptoms_count	0.487903

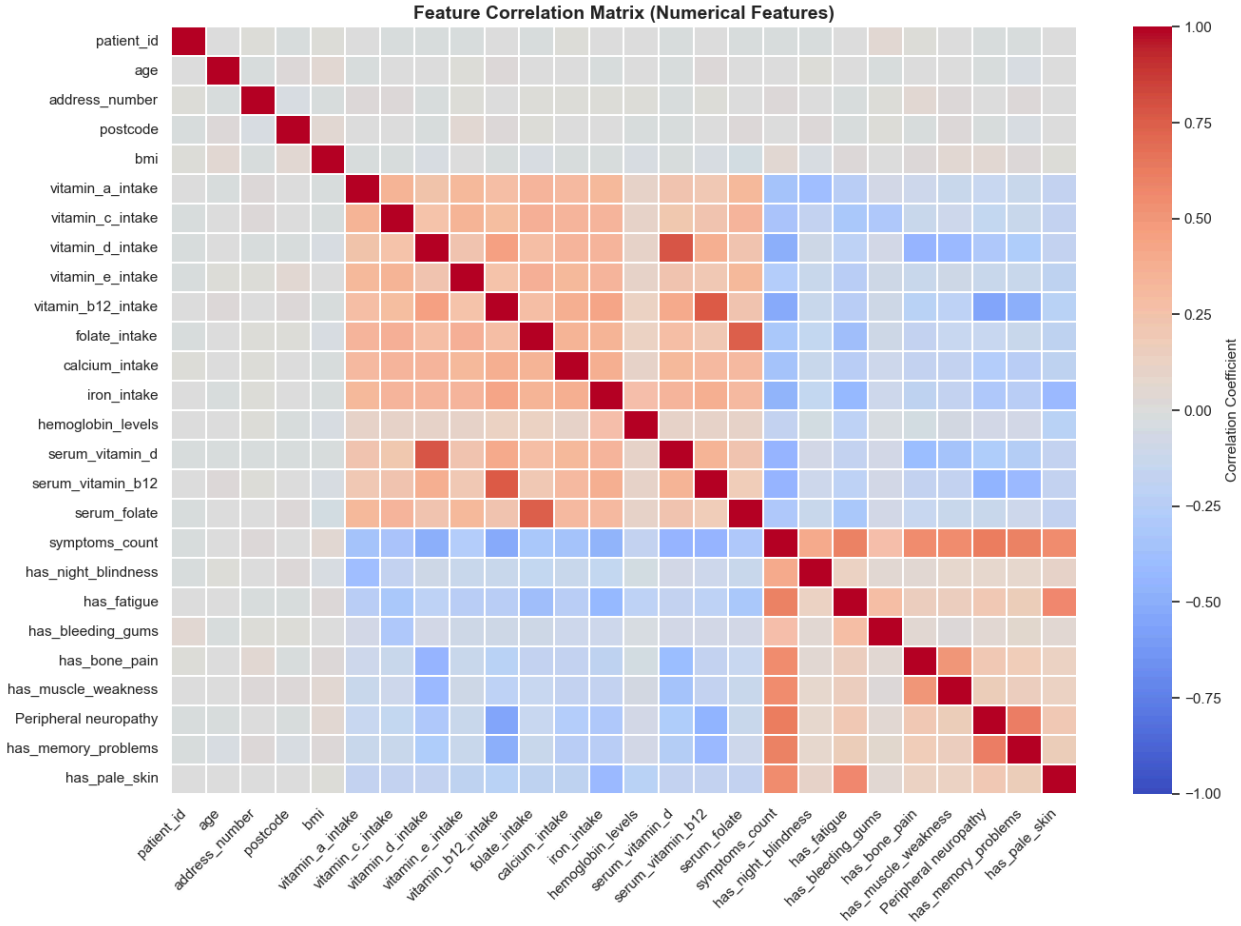
```
Pairs with |correlation| >= 0.80: 0
```

feature_1	feature_2	abs_corr

In [57]:
```
# Correlation & Multicollinearity Matrix (reliable plotting)
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
# Select numeric features only
num_cols = df.select_dtypes(include=["number"]).columns.tolist()

# Remove target if numeric by any chance
if "target_name" in globals() and target_name in num_cols:
    num_cols.remove(target_name)

print(f"Numeric feature count: {len(num_cols)}")

if len(num_cols) >= 2:
    corr = df[num_cols].corr()

    # Plot correlation matrix
    plt.figure(figsize=(14, 10))
    sns.heatmap(
        corr,
        cmap="coolwarm",
        vmin=-1, vmax=1,
        center=0,
        linewidths=0.3,
        cbar_kws={"label": "Correlation Coefficient"}
    )
    plt.title("Feature Correlation Matrix (Numerical Features)", fontsize=14,
    plt.xticks(rotation=45, ha="right")
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()

    # Multicollinearity check: high-correlation pairs
    corr_abs = corr.abs()
    upper = corr_abs.where(np.triu(np.ones(corr_abs.shape), k=1).astype(bool))

    high_corr_pairs = (
        upper.stack()
             .reset_index()
    )
    high_corr_pairs.columns = ["feature_1", "feature_2", "abs_corr"]
    high_corr_pairs = high_corr_pairs.sort_values("abs_corr", ascending=False)

    threshold = 0.80
    flagged = high_corr_pairs[high_corr_pairs["abs_corr"] >= threshold]

    print(f"\nTop correlated pairs:")
    display(high_corr_pairs.head(15))

    print(f"\nPairs with |correlation| >= {threshold}: {len(flagged)}")
    if len(flagged) > 0:
        display(flagged)
    else:
        print("No feature pairs exceed threshold.")
else:
    print("Not enough numerical columns to compute correlation matrix.")
```

Numeric feature count: 26



Top correlated pairs:

	feature_1	feature_2	abs_corr
160	vitamin_d_intake	serum_vitamin_d	0.785847
194	vitamin_b12_intake	serum_vitamin_b12	0.761144
210	folate_intake	serum_folate	0.753929
294	symptoms_count	Peripheral neuropathy	0.629869
322	Peripheral neuropathy	has_memory_problems	0.612786
295	symptoms_count	has_memory_problems	0.602249
290	symptoms_count	has_fatigue	0.601908
309	has_fatigue	has_pale_skin	0.567190
292	symptoms_count	has_bone_pain	0.554015
293	symptoms_count	has_muscle_weakness	0.547230
296	symptoms_count	has_pale_skin	0.544214
202	vitamin_b12_intake	Peripheral neuropathy	0.540530
196	vitamin_b12_intake	symptoms_count	0.523432
315	has_bone_pain	has_muscle_weakness	0.503144
163	vitamin_d_intake	symptoms_count	0.487903

```
Pairs with |correlation| >= 0.8: 0
No feature pairs exceed threshold.
```

In [58]:

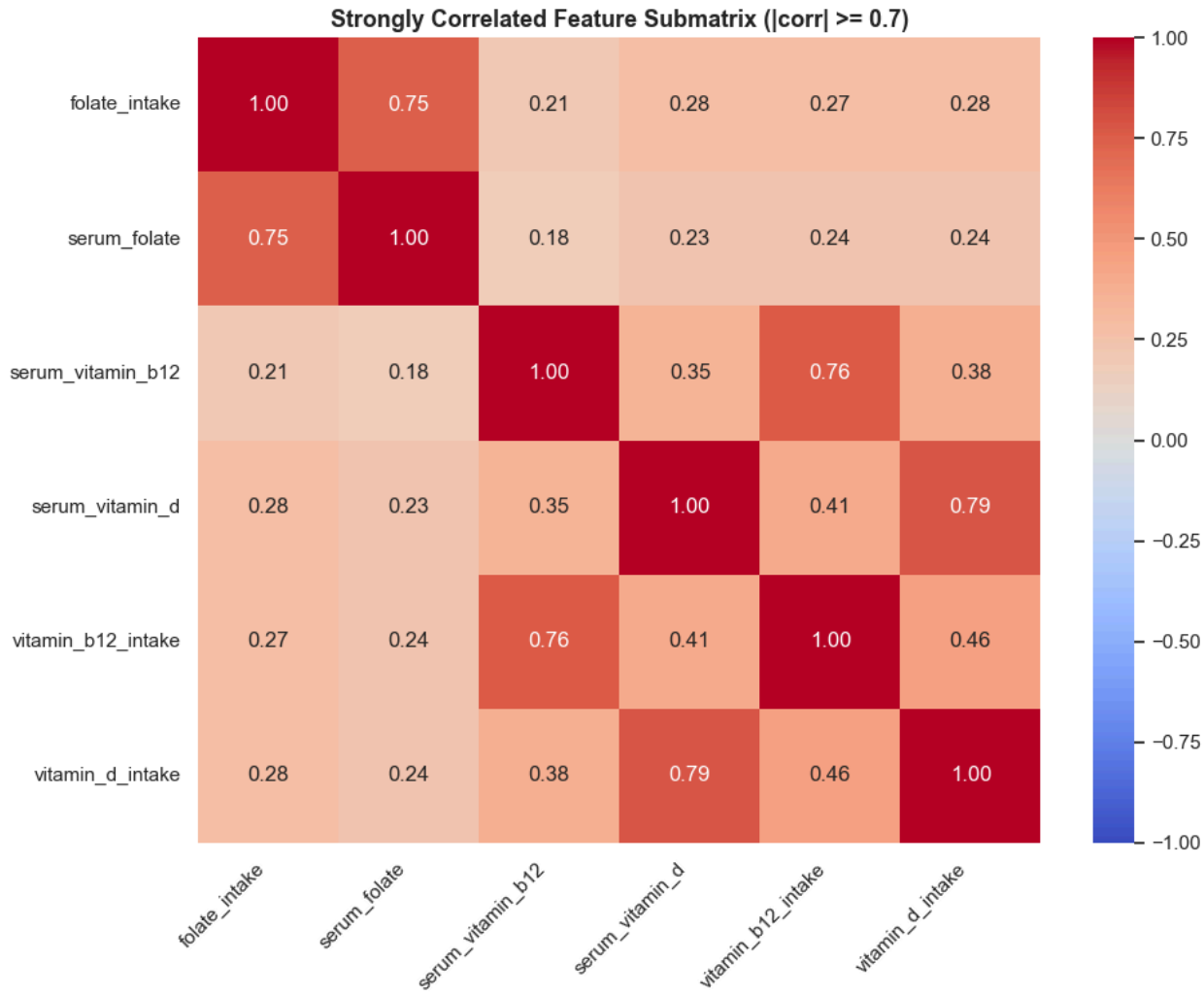
```python
# Robust strong-correlation submatrix plotting
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Recompute from corr to avoid dependency issues
corr_abs = corr.abs()
upper = corr_abs.where(np.triu(np.ones(corr_abs.shape), k=1).astype(bool))
pairs = upper.stack().reset_index()
pairs.columns = ["feature_1", "feature_2", "abs_corr"]
pairs = pairs.sort_values("abs_corr", ascending=False)

# Start with strict threshold, then relax if empty
threshold = 0.80
flagged = pairs[pairs["abs_corr"] >= threshold]

if flagged.empty:
    print(f"No pairs found with |corr| >= {threshold}. Trying 0.70 ...")
    threshold = 0.70
    flagged = pairs[pairs["abs_corr"] >= threshold]

if flagged.empty:
    print(f"No pairs found with |corr| >= {threshold} either.")
```

```
    print("Top 10 correlations in your data:")
    display(pairs.head(10))
else:
    print(f"Using threshold {threshold}. Found {len(flagged)} pairs.")
    display(flagged.head(20))

    strong_features = sorted(set(flagged["feature_1"]).union(set(flagged["feat
    print(f"Strong feature count: {len(strong_features)}")

    if len(strong_features) >= 2:
        plt.figure(figsize=(10, 8))
        sns.heatmap(
            corr.loc[strong_features, strong_features],
            annot=True, fmt=".2f",
            cmap="coolwarm", vmin=-1, vmax=1, center=0
        )
        plt.title(f"Strongly Correlated Feature Submatrix (|corr| >= {threshol
        plt.xticks(rotation=45, ha="right")
        plt.yticks(rotation=0)
        plt.tight_layout()
        plt.show()
    else:
        print("Not enough strongly correlated features to plot a submatrix.")
```

```
No pairs found with |corr| >= 0.8. Trying 0.70 ...
Using threshold 0.7. Found 3 pairs.
```

	feature_1	feature_2	abs_corr
160	vitamin_d_intake	serum_vitamin_d	0.785847
194	vitamin_b12_intake	serum_vitamin_b12	0.761144
210	folate_intake	serum_folate	0.753929

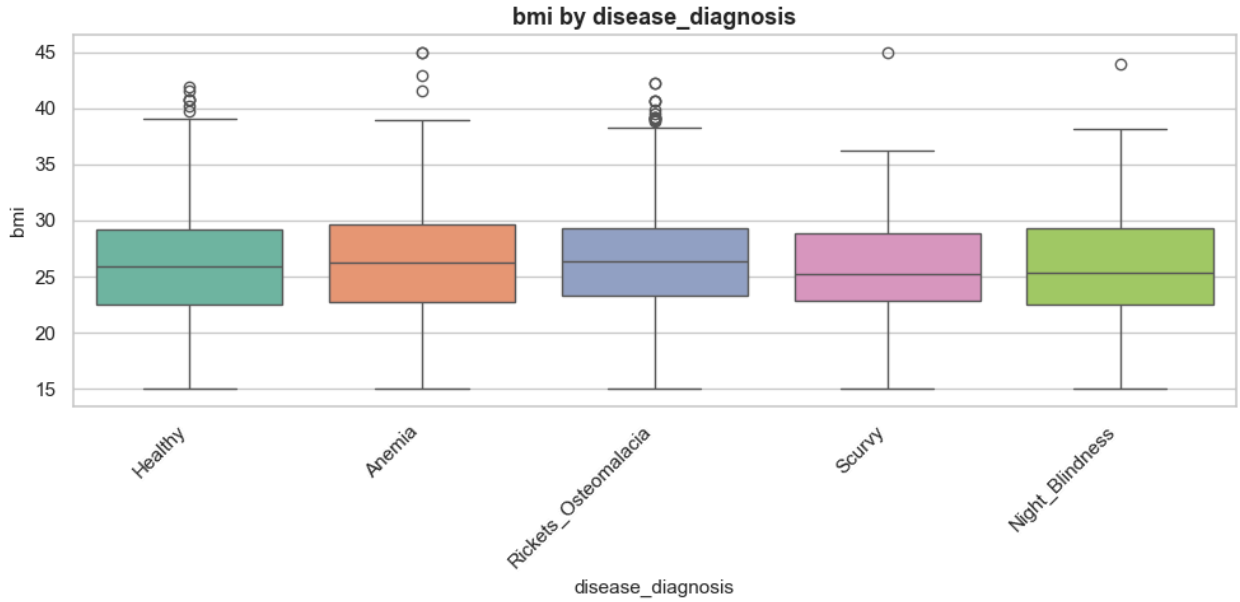
```
Strong feature count: 6
```

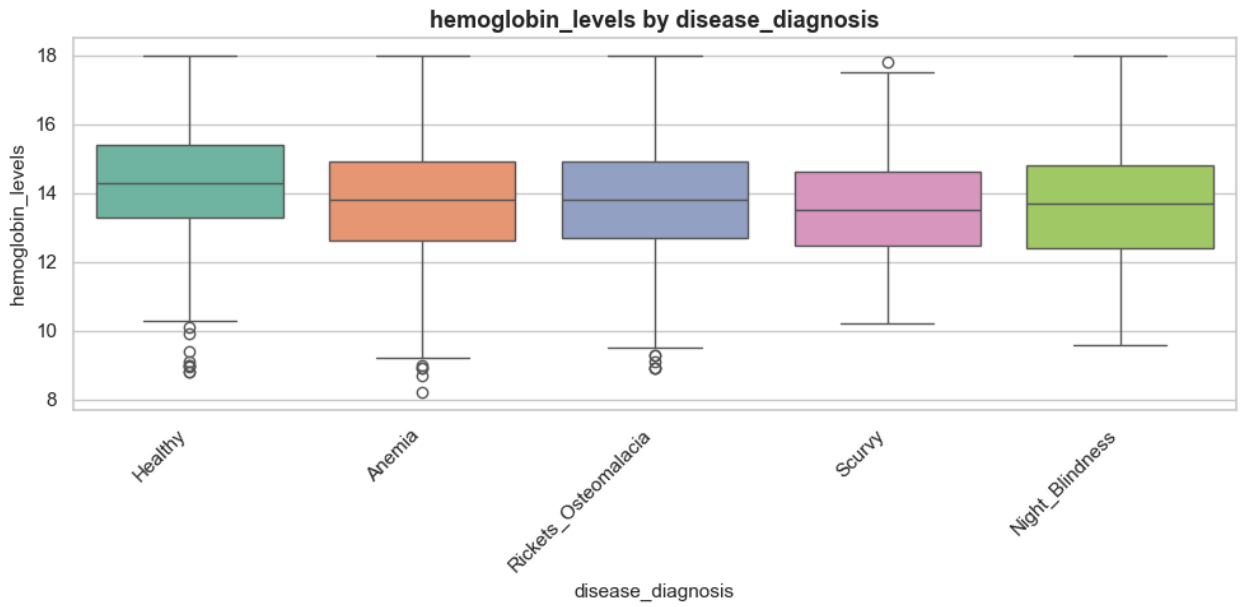


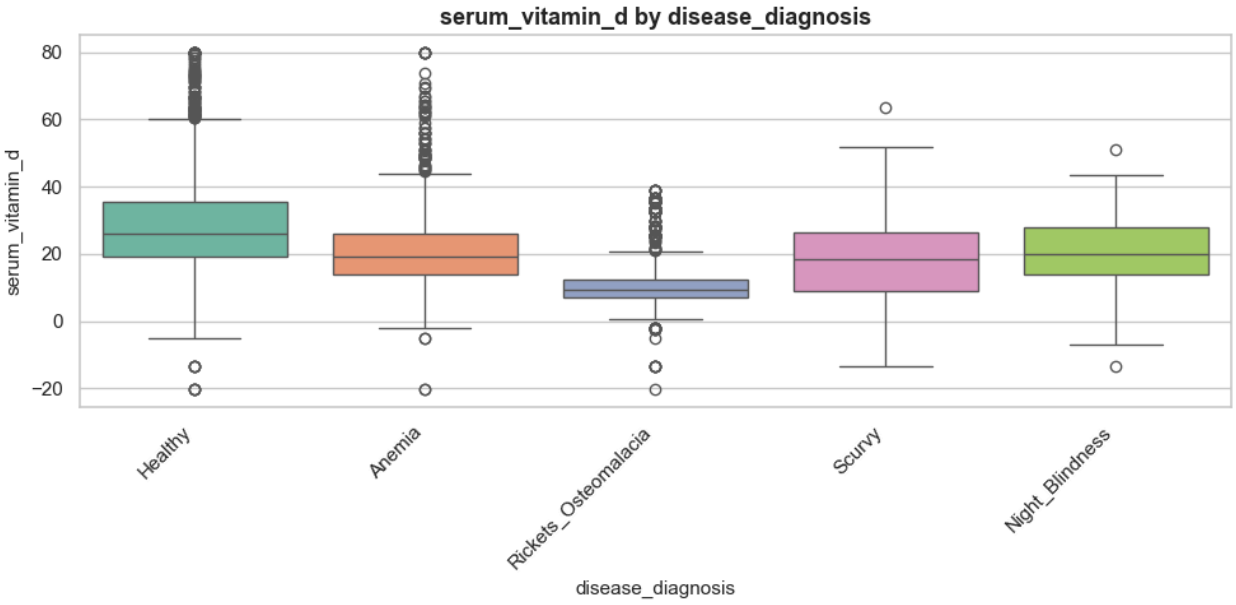
In [59]:

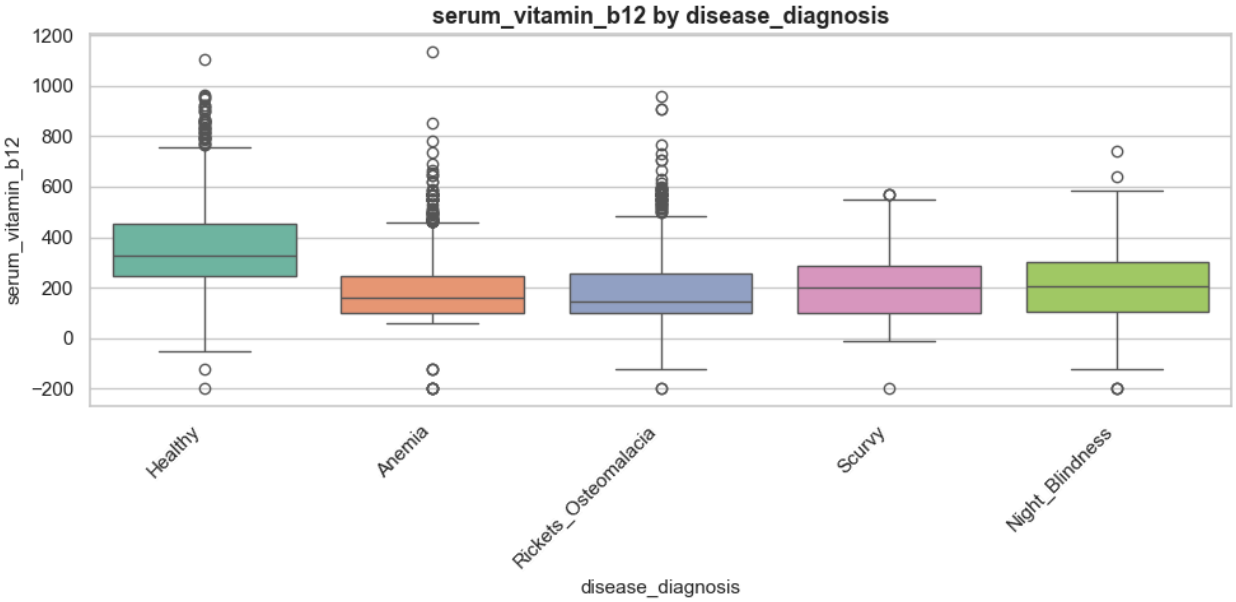
```
# ============================================
# Target relationship with selected features
# ============================================
# Choose a few meaningful numeric indicators if present
candidate_numeric = [
    "bmi", "hemoglobin_levels", "serum_vitamin_d", "serum_vitamin_b12",
    "serum_folate", "vitamin_d_intake", "vitamin_b12_intake", "folate_intake",
    "symptoms_count", "age"
]
candidate_numeric = [c for c in candidate_numeric if c in df.columns]

for col in candidate_numeric[:8]:
    plt.figure(figsize=(10, 5))
    sns.boxplot(data=df, x=target_col, y=col, palette="Set2")
    plt.title(f"{col} by {target_col}")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

# Categorical relation examples
candidate_cats = ["gender", "smoking_status", "alcohol_consumption", "diet_typ
candidate_cats = [c for c in candidate_cats if c in df.columns]
```

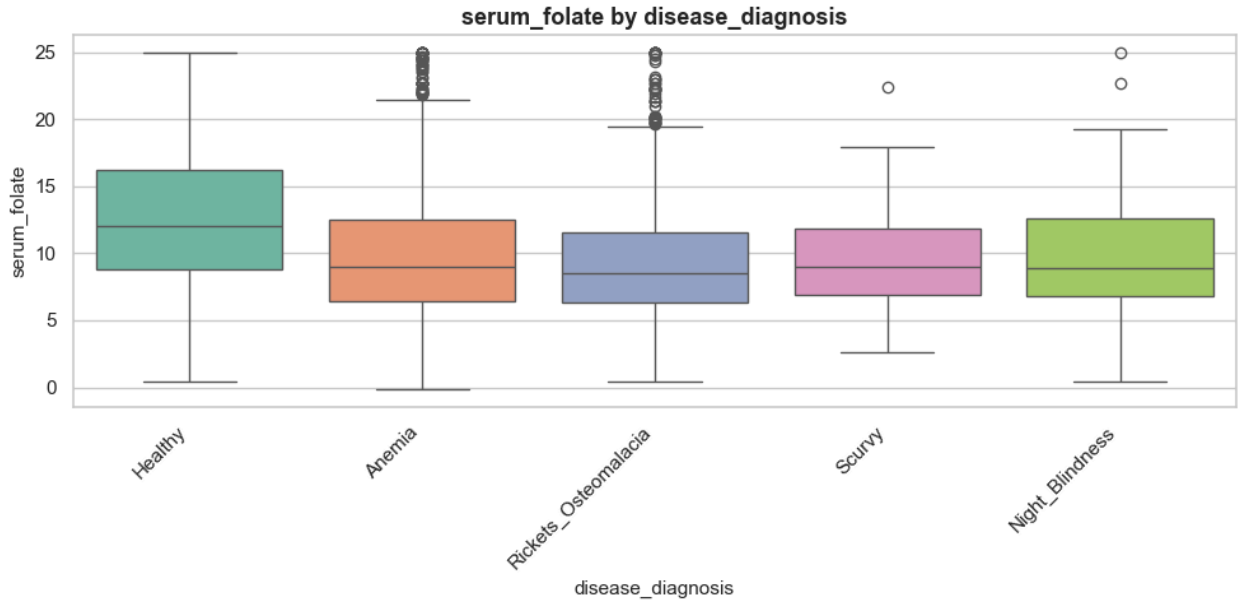
```python
for col in candidate_cats:
    cross_tab = pd.crosstab(df[col], df[target_col], normalize="index") * 100
    display(cross_tab.round(2))
```

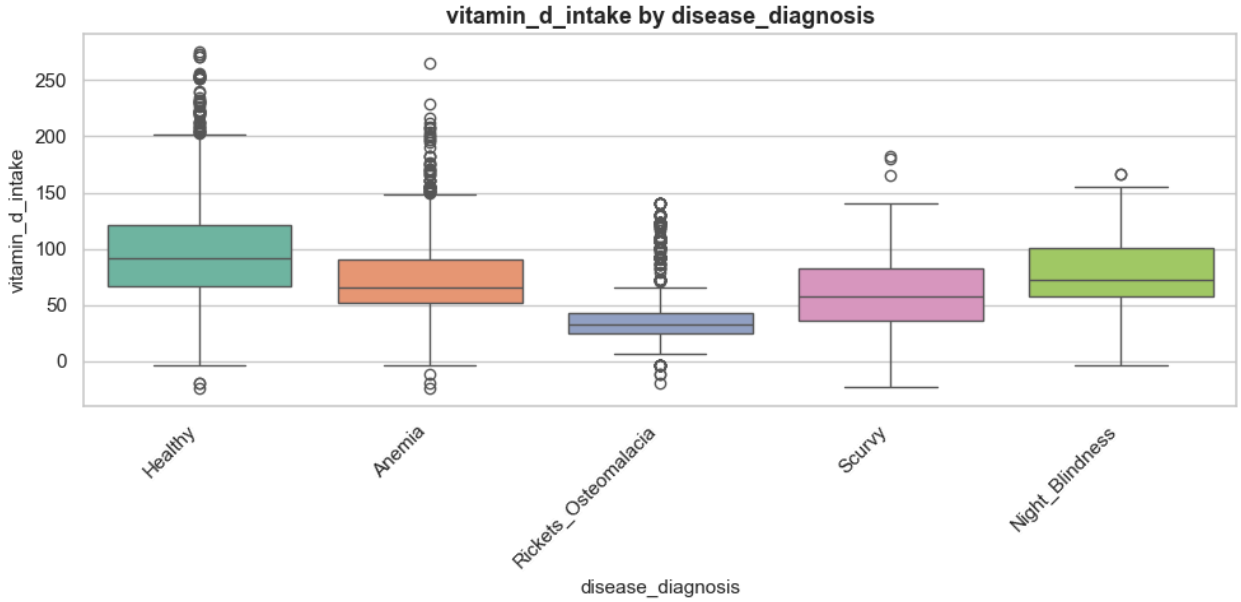


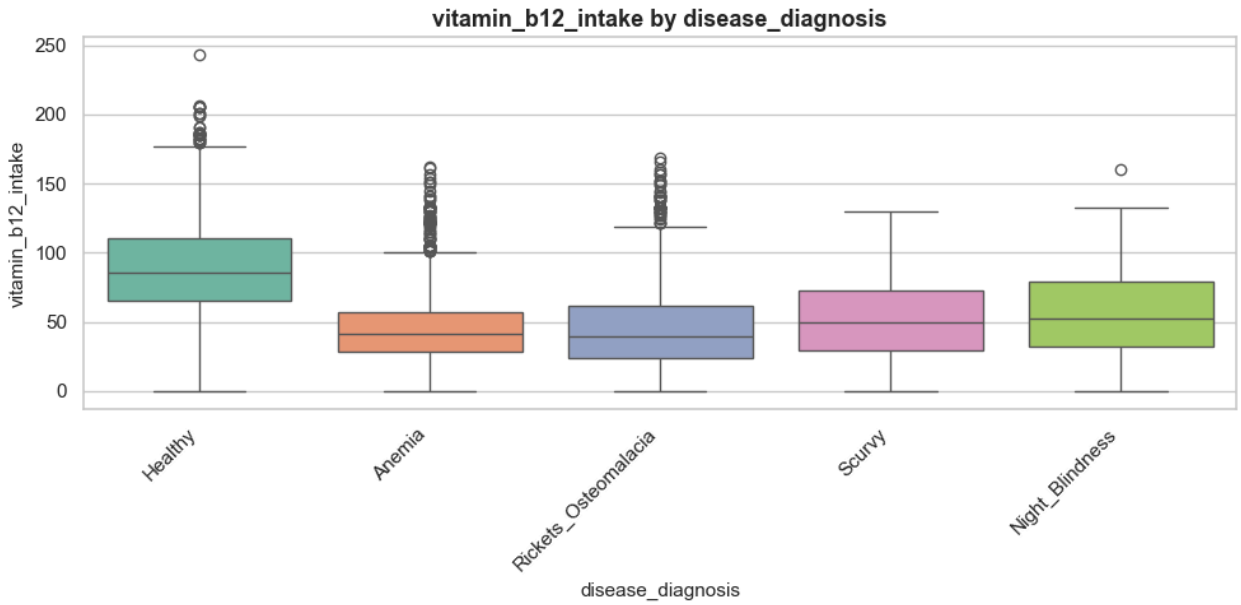


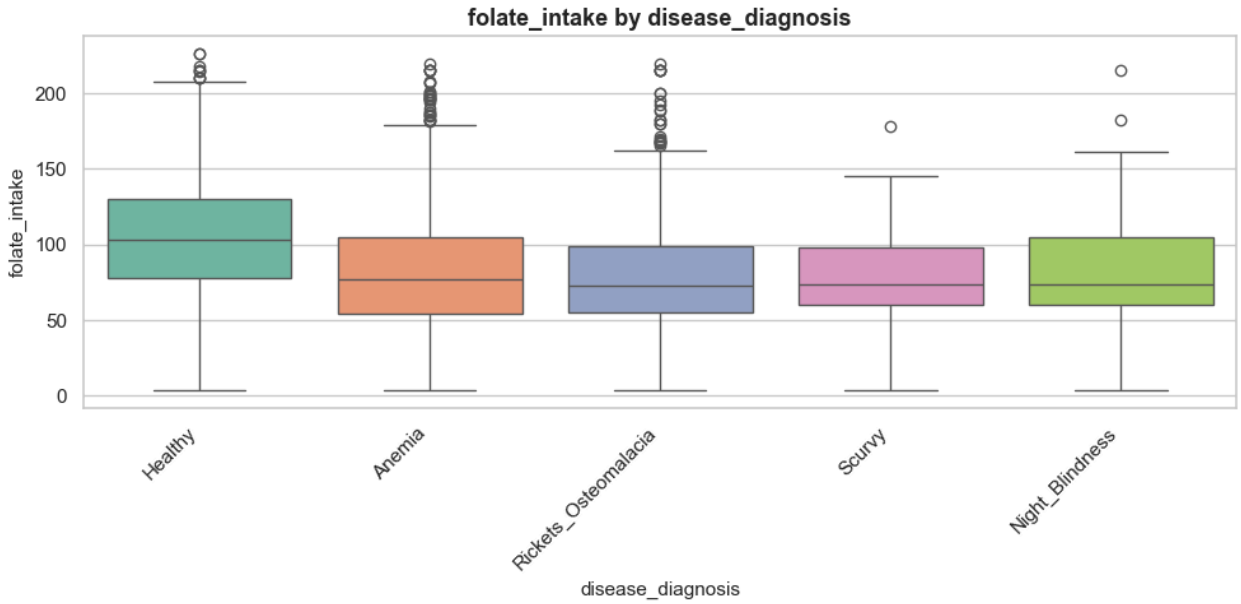

serum_vitamin_d by disease_diagnosis
serum_vitamin_d
80
60
40
20
0
−20
Healthy
Anemia
Rickets_Osteomalacia
Scurvy
Night_Blindness
disease_diagnosis


serum_vitamin_b12 by disease_diagnosis
serum_vitamin_b12
1200
1000
800
600
400
200
0
−200
Healthy
Anemia
Rickets_Osteomalacia
Scurvy
Night_Blindness
disease_diagnosis


serum_folate by disease_diagnosis
serum_folate
25
20
15
10
5
0
Healthy
Anemia
Rickets_Osteomalacia
Scurvy
Night_Blindness
disease_diagnosis


vitamin_d_intake by disease_diagnosis
vitamin_d_intake
250
200
150
100
50
0
Healthy
Anemia
Rickets_Osteomalacia
Scurvy
Night_Blindness
disease_diagnosis





disease_diagnosis gender	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Scur
Female	31.33	36.82	4.38	24.51	2.
Male	30.92	36.32	4.89	24.76	3.

disease_diagnosis smoking_status	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Scur
Current	34.12	35.95	3.93	23.25	2.
Former	29.07	38.48	4.12	25.37	2.
Never	30.68	35.83	5.09	25.58	2.
Unknown	25.00	41.67	5.56	22.22	5.

disease_diagnosis alcohol_consumption	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Sc
Heavy	30.22	35.95	4.85	25.89	
Moderate	30.57	36.11	4.31	26.00	
Unknown	19.61	37.25	9.80	23.53	

disease_diagnosis diet_type	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Scur
Omnivore	21.31	54.93	3.67	18.00	2.
Pescatarian	25.96	53.06	5.55	12.59	2.
Unknown	29.55	29.55	13.64	18.18	9.
Vegan	43.32	5.98	4.30	42.98	3.
Vegetarian	33.36	35.15	3.80	24.27	3.

disease_diagnosis exercise_level	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Scur
Active	30.79	39.08	3.70	24.06	2.
Light	32.61	35.71	4.83	24.39	2.
Moderate	31.11	34.91	4.41	26.39	3.
Sedentary	30.66	37.58	3.74	24.78	3.
Unknown	34.75	31.36	10.17	18.64	5.

disease_diagnosis sun_exposure_type	Anemia	Healthy	Night_Blindness	Rickets_Osteomalacia	Scu
High	42.69	41.32	6.10	6.59	3
Low	12.13	29.68	1.90	54.16	2
Moderate	37.89	39.53	4.59	14.67	3
Unknown	30.73	29.05	11.73	24.02	4

In [60]:
```
EDA_2_insights = """
## EDA Findings and Implications for Modeling

### Key observations
1. **Target is multiclass** and shows class imbalance (majority and minority c
2. Clinical variables (e.g., serum vitamin markers, hemoglobin, symptoms count
3. Lifestyle/diet variables (diet type, sun exposure, smoking/alcohol/exercise
4. Several columns are **identifiers/PII** (`customer_id`, `patient_id`, `full
5. Some high-cardinality/free-text-like fields (e.g., `symptoms_list`) may int

### Data preparation decisions for next notebook
- Drop identifier/PII fields and irrelevant leakage-prone fields.
- Keep structured clinical, lifestyle, and demographic predictors.
- Handle missing values with robust imputation (median for numeric, most frequ
- Use encoding for categorical features and scaling for scale-sensitive models
- Use stratified splits and class-sensitive metrics.

### Experiment hypotheses
- **H1:** Tree-based models will outperform linear baselines due to non-linear
- **H2:** Class imbalance handling (e.g., class weighting) will improve minori
- **H3:** Structured feature engineering (intake + serum + symptom interactior
"""
```

In [61]:
```
print_tile(size="h3", key='EDA_2_Insights', value=EDA_2_insights)
```

EDA Findings and Implications for Modeling ### Key observations 1. **Target is multiclass** and shows class imbalance (majority and minority diagnosis groups differ noticeably). 2. Clinical variables (e.g., serum vitamin markers, hemoglobin, symptoms count) show useful signal for classification. 3. Lifestyle/diet variables (diet type, sun exposure, smoking/alcohol/exercise) are potentially predictive and should be retained. 4. Several columns are **identifiers/PII** (`customer_id`, `patient_id`, `full_name`, `email`, `phone`, detailed address fields) and should be excluded from modeling. 5. Some high-cardinality/free-text-like fields (e.g., `symptoms_list`) may introduce leakage/noise if not handled carefully. ### Data preparation decisions for next notebook - Drop identifier/PII fields and irrelevant leakage-prone fields. - Keep structured clinical, lifestyle, and demographic predictors. - Handle missing values with robust imputation (median for numeric, most frequent for categorical). - Use encoding for categorical features and scaling for scale-sensitive models. - Use stratified splits and class-sensitive metrics. ### Experiment hypotheses - **H1:** Tree-based models will outperform linear baselines due to non-linear relations between symptoms and biomarkers. - **H2:** Class imbalance handling (e.g., class weighting) will improve minority-class recall and macro-F1. - **H3:** Structured feature engineering (intake + serum + symptom interactions) will improve discriminative performance.

C.2 Explore Feature of Interest `serum_vitamin_d`

> You can add more cells in this section

In [62]:
```
feature_1_name = "serum_vitamin_d"

print("Feature exists:", feature_1_name in df.columns)
print("Missing values:", df[feature_1_name].isna().sum())
display(df[feature_1_name].describe().to_frame("summary"))

# Distribution histogram
hist_chart = (
    alt.Chart(df)
    .mark_bar()
    .encode(
```

```
        x=alt.X(f"{feature_1_name}:Q", bin=alt.Bin(maxbins=35), title=feature_
        y=alt.Y("count():Q", title="Frequency"),
        tooltip=[alt.Tooltip("count():Q", title="Count")]
    )
    .properties(title=f"Distribution of {feature_1_name}", width=700, height=3
)
hist_chart
```

```
Feature exists: True
Missing values: 0
```

	summary
count	5800.000000
mean	21.214884
std	13.687205
min	-20.412610
25%	11.700000
50%	18.800000
75%	27.941359
max	80.000000

Out[62]:

In [63]:

```
# Boxplot-like summary with Altair (facet by target)
sample_df = df[[feature_1_name, target_name]].dropna().copy()

box_chart = (
    alt.Chart(sample_df)
    .mark_boxplot(extent='min-max')
    .encode(
        x=alt.X(f"{target_name}:N", sort="-y", title="Disease Diagnosis"),
        y=alt.Y(f"{feature_1_name}:Q", title=feature_1_name),
        color=alt.Color(f"{target_name}:N", legend=None)
    )
    .properties(title=f"{feature_1_name} by {target_name}", width=700, height=
)
box_chart
```

Out[63]:

In [64]:

```
feature_1_insights = """
Feature selected: serum_vitamin_d

Findings:
1) The distribution shows spread across low to high serum vitamin D values, wi
2) Missing values should be handled carefully to avoid bias in model training.
3) Class-conditional distributions suggest this feature may carry useful signa
4) Clinical plausibility supports this feature as important for deficiency-rel
```

```
Modeling implications:
- Keep this feature for all experiments.
- Apply robust imputation (median) if missing.
- Tree-based models can naturally handle non-linear thresholds; linear models
"""
```

In [65]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

feature_1_insights

Feature selected: serum_vitamin_d Findings: 1) The distribution shows spread across low to high serum vitamin D values, with potential tails/outliers. 2) Missing values should be handled carefully to avoid bias in model training. 3) Class-conditional distributions suggest this feature may carry useful signal for distinguishing disease diagnosis classes. 4) Clinical plausibility supports this feature as important for deficiency-related conditions. Modeling implications: - Keep this feature for all experiments. - Apply robust imputation (median) if missing. - Tree-based models can naturally handle non-linear thresholds; linear models may need scaling and interaction support.

C.3 Explore Feature of Interest `Symptoms Count`

> You can add more cells in this section

In [66]:
```
feature_2_name = "symptoms_count"

print("Feature exists:", feature_2_name in df.columns)
print("Missing values:", df[feature_2_name].isna().sum())
display(df[feature_2_name].describe().to_frame("summary"))

symptom_hist = (
    alt.Chart(df)
    .mark_bar()
    .encode(
        x=alt.X(f"{feature_2_name}:Q", bin=alt.Bin(maxbins=25), title=feature_
        y=alt.Y("count():Q", title="Frequency"),
        tooltip=[alt.Tooltip("count():Q", title="Count")]
    )
    .properties(title=f"Distribution of {feature_2_name}", width=700, height=3
)
symptom_hist
```

```
Feature exists: True
Missing values: 0
```

	summary
count	5800.000000
mean	1.868276
std	1.806913
min	0.000000
25%	0.000000
50%	2.000000
75%	3.000000
max	9.000000

Out[66]:

In [67]:
```
symptom_target_chart = (
    alt.Chart(df[[feature_2_name, target_name]].dropna())
    .mark_boxplot(extent='min-max')
    .encode(
        x=alt.X(f"{target_name}:N", sort="-y", title="Disease Diagnosis"),
        y=alt.Y(f"{feature_2_name}:Q", title=feature_2_name),
        color=alt.Color(f"{target_name}:N", legend=None)
    )
    .properties(title=f"{feature_2_name} by {target_name}", width=700, height=
)
symptom_target_chart
```

Out[67]:

In [68]:
```
feature_2_insights = """
Feature selected: symptoms_count

Findings:
1) Symptoms count captures disease burden intensity and is intuitively relevar
2) Distribution and class-level differences indicate good predictive potential
3) Any missing values should be imputed consistently; if sparse, median imputa
4) This feature may interact with serum markers and intake variables to improv

Modeling implications:
- Retain as a core predictor in baseline and advanced experiments.
- Test whether models with interactions/non-linearity (e.g., Random Forest, Gr
- Use per-class recall to ensure this feature supports minority-class sensitiv
"""
```

In [69]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

feature_2_insights

Feature selected: symptoms_count Findings: 1) Symptoms count captures disease burden intensity and is intuitively relevant for diagnosis. 2) Distribution and class-level differences indicate good predictive potential. 3) Any missing values should be imputed consistently; if sparse, median imputation is appropriate. 4) This feature may interact with serum markers and intake variables to improve discrimination between classes. Modeling implications: - Retain as a core predictor in baseline and advanced experiments. - Test whether models with interactions/non-linearity (e.g., Random Forest, Gradient Boosting) improve performance. - Use per-class recall to ensure this feature supports minority-class sensitivity.

C.4 Explore Feature of Interest `diet type`

> You can add more cells in this section

In [70]:
```
feature_3_name = "diet_type"

print("Feature exists:", feature_3_name in df.columns)
print("Missing values:", df[feature_3_name].isna().sum())
print("Unique categories:", df[feature_3_name].nunique(dropna=False))
display(df[feature_3_name].value_counts(dropna=False).to_frame("count"))

diet_counts = (
    df[feature_3_name]
    .value_counts(dropna=False)
    .reset_index()
)
diet_counts.columns = [feature_3_name, "count"]

diet_chart = (
    alt.Chart(diet_counts)
    .mark_bar()
    .encode(
        x=alt.X(f"{feature_3_name}:N", sort="-y", title=feature_3_name),
        y=alt.Y("count:Q", title="Count"),
        tooltip=[feature_3_name, "count"]
    )
    .properties(title=f"Category Distribution: {feature_3_name}", width=700, h
)
diet_chart
```

```
Feature exists: True
Missing values: 129
Unique categories: 6
```

diet_type	count
Vegan	1489
Pescatarian	1406
Omnivore	1389
Vegetarian	1343
NaN	129
Unknown	44

Out[70]:

In [71]:
```
# Normalized class distribution by diet_type
diet_target = pd.crosstab(df[feature_3_name], df[target_name], normalize="inde
diet_target_long = diet_target.melt(id_vars=feature_3_name, var_name=target_na

diet_target_chart = (
    alt.Chart(diet_target_long)
    .mark_bar()
    .encode(
        x=alt.X(f"{feature_3_name}:N", title=feature_3_name),
        y=alt.Y("proportion:Q", stack="normalize", title="Proportion"),
        color=alt.Color(f"{target_name}:N", title="Diagnosis"),
        tooltip=[feature_3_name, target_name, alt.Tooltip("proportion:Q", form
    )
    .properties(title=f"{target_name} composition within each {feature_3_name}
)
diet_target_chart
```

Out[71]:

In [72]:
```
feature_3_insights = """
Feature selected: diet_type

Findings:
1) diet_type is a meaningful lifestyle variable with potential links to micron
2) Category frequencies should be checked for rare classes; rare categories ca
3) Class composition across diet types suggests potential association with dia
4) As a categorical feature, it should be encoded appropriately (e.g., One-Hot

Modeling implications:
- Keep this feature in all experiments.
- Handle missing values using most frequent category.
- Combine with intake and serum variables to test interaction effects.
"""

# Template compatibility fix:
feature_n_insights = feature_3_insights
```

In [73]:
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_3_insights', value=feature_3_insights)
```

feature_3_insights

Feature selected: diet_type Findings: 1) diet_type is a meaningful lifestyle variable with potential links to micronutrient intake patterns. 2) Category frequencies should be checked for rare classes; rare categories can increase variance in estimates. 3) Class composition across diet types suggests potential association with diagnosis categories. 4) As a categorical feature, it should be encoded appropriately (e.g., One-Hot Encoding) in model pipelines. Modeling implications: - Keep this feature in all experiments. - Handle missing values using most frequent category. - Combine with intake and serum variables to test interaction effects.

C.n Explore Feature of Interest `\<put feature name here\>`

> You can add more cells related to other feeatures in this section

In [74]:
```
# Additional in-depth EDA: data quality, missingness, duplicates, leakage/PII

# 1) Shape and data types
print("Dataset shape:", df.shape)
display(df.dtypes.to_frame("dtype"))

# 2) Missing values summary
missing_df = (
    df.isna().sum()
      .reset_index()
)
missing_df.columns = ["feature", "missing_count"]
missing_df["missing_pct"] = (missing_df["missing_count"] / len(df) * 100).rour
missing_df = missing_df.sort_values("missing_pct", ascending=False)
display(missing_df.head(20))

missing_chart = (
    alt.Chart(missing_df.head(20))
    .mark_bar()
    .encode(
        x=alt.X("missing_pct:Q", title="% Missing"),
        y=alt.Y("feature:N", sort="-x", title="Feature"),
        tooltip=["feature", "missing_count", "missing_pct"]
    )
    .properties(title="Top 20 Features by Missingness", width=700, height=450)
```

```
)
missing_chart
```

```
Dataset shape: (5800, 45)
```

	dtype
customer_id	object
patient_id	int64
gender	object
age	int64
birth_country	object
full_name	object
address	object
address_number	int64
street_name	object
street_suffix	object
city	object
postcode	int64
state	object
email	object
phone	object
bmi	float64
smoking_status	object
alcohol_consumption	object
exercise_level	object
diet_type	object
sun_exposure_type	object
income_band	object
latitude_band	object
vitamin_a_intake	float64
vitamin_c_intake	float64
vitamin_d_intake	float64
vitamin_e_intake	float64
vitamin_b12_intake	float64
folate_intake	float64
calcium_intake	float64
iron_intake	float64

	dtype
hemoglobin_levels	float64
serum_vitamin_d	float64
serum_vitamin_b12	float64
serum_folate	float64
symptoms_count	int64
has_night_blindness	int64
has_fatigue	int64
has_bleeding_gums	int64
has_bone_pain	int64
has_muscle_weakness	int64
Peripheral neuropathy	int64
has_memory_problems	int64
has_pale_skin	int64
disease_diagnosis	object

	feature	missing_count	missing_pct
17	alcohol_consumption	1952	33.66
18	exercise_level	208	3.59
22	latitude_band	174	3.00
16	smoking_status	169	2.91
21	income_band	158	2.72
19	diet_type	129	2.22
20	sun_exposure_type	117	2.02
32	serum_vitamin_d	0	0.00
28	folate_intake	0	0.00
29	calcium_intake	0	0.00
30	iron_intake	0	0.00
31	hemoglobin_levels	0	0.00
33	serum_vitamin_b12	0	0.00
26	vitamin_e_intake	0	0.00
34	serum_folate	0	0.00
35	symptoms_count	0	0.00
36	has_night_blindness	0	0.00
37	has_fatigue	0	0.00
38	has_bleeding_gums	0	0.00
39	has_bone_pain	0	0.00

Out[74]:

In [75]:
```
# 3) Duplicates
duplicate_rows = df.duplicated().sum()
duplicate_pct = round((duplicate_rows / len(df)) * 100, 3)
print(f"Duplicate rows: {duplicate_rows} ({duplicate_pct}%)")

# 4) Potential ID/PII and leakage fields
possible_pii = [
    "customer_id", "patient_id", "full_name", "address", "address_number",
    "street_name", "street_suffix", "email", "phone"
]
possible_leakage = ["symptoms_list"]

pii_present = [c for c in possible_pii if c in df.columns]
leakage_present = [c for c in possible_leakage if c in df.columns]
```

```
print("PII/Identifier columns present:", pii_present)
print("Potential leakage/high-cardinality text columns present:", leakage_pres

# 5) Correlation quick view for numerical variables
num_cols = df.select_dtypes(include=["number"]).columns.tolist()
if len(num_cols) >= 2:
    corr_long = (
        df[num_cols]
        .corr()
        .abs()
        .stack()
        .reset_index()
    )
    corr_long.columns = ["feature_1", "feature_2", "abs_corr"]
    corr_long = corr_long[corr_long["feature_1"] != corr_long["feature_2"]]
    corr_long["pair_key"] = corr_long.apply(lambda r: "-".join(sorted([r["feat
    corr_long = corr_long.drop_duplicates("pair_key").sort_values("abs_corr",
    display(corr_long[["feature_1", "feature_2", "abs_corr"]].head(15))
else:
    print("Not enough numeric columns for correlation analysis.")
```

```
Duplicate rows: 0 (0.0%)
PII/Identifier columns present: ['customer_id', 'patient_id', 'full_name', 'add
ress', 'address_number', 'street_name', 'street_suffix', 'email', 'phone']
Potential leakage/high-cardinality text columns present: []
```

	feature_1	feature_2	abs_corr
196	vitamin_d_intake	serum_vitamin_d	0.785847
249	vitamin_b12_intake	serum_vitamin_b12	0.761144
276	folate_intake	serum_folate	0.753929
465	symptoms_count	Peripheral neuropathy	0.629869
622	Peripheral neuropathy	has_memory_problems	0.612786
466	symptoms_count	has_memory_problems	0.602249
461	symptoms_count	has_fatigue	0.601908
519	has_fatigue	has_pale_skin	0.567190
463	symptoms_count	has_bone_pain	0.554015
464	symptoms_count	has_muscle_weakness	0.547230
467	symptoms_count	has_pale_skin	0.544214
257	vitamin_b12_intake	Peripheral neuropathy	0.540530
251	vitamin_b12_intake	symptoms_count	0.523432
568	has_bone_pain	has_muscle_weakness	0.503144
199	vitamin_d_intake	symptoms_count	0.487903

In [76]:
```
feature_n_insights = """
```

```
Additional EDA (data quality and risk assessment) identified key consideration

1) Missingness:
   - Several features may contain missing values; these must be systematically
   - Numeric features should use median imputation, categorical features shoul

2) Duplicates:
   - Duplicate rows were quantified and should be removed in preparation if th

3) PII and identifier fields:
   - Columns such as names, contact details, and exact address components are
   - They should be removed for privacy, ethical compliance, and to reduce ove

4) Leakage/high-cardinality fields:
   - Text-like and highly granular fields (e.g., symptoms_list) can cause leak
   - These require careful handling or exclusion depending on experiment desig

5) Correlation structure:
   - Some numerical variables are related; this can affect linear models and i
   - Model pipelines should be robust to multicollinearity and non-linear patt

Overall conclusion:
The dataset appears suitable for multiclass predictive modeling after discipli
Next steps are to apply reproducible preprocessing, remove risky columns, stra
"""
```

In [77]:

```python
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)
```

feature_n_insights

Additional EDA (data quality and risk assessment) identified key considerations for reliable modeling: 1) Missingness: - Several features may contain missing values; these must be systematically imputed. - Numeric features should use median imputation, categorical features should use mode/most_frequent. 2) Duplicates: - Duplicate rows were quantified and should be removed in preparation if they represent repeated records. 3) PII and identifier fields: - Columns such as names, contact details, and exact address components are not suitable model inputs. - They should be removed for privacy, ethical compliance, and to reduce overfitting risk. 4) Leakage/high-cardinality fields: - Text-like and highly granular fields (e.g., symptoms_list) can cause leakage or unstable encoding. - These require careful handling or exclusion depending on experiment design. 5) Correlation structure: - Some numerical variables are related; this can affect linear models and interpretation. - Model pipelines should be robust to multicollinearity and non-linear patterns. Overall conclusion: The dataset appears suitable for multiclass predictive modeling after disciplined preparation. Next steps are to apply reproducible preprocessing, remove risky columns, stratify splits, and evaluate models using class-sensitive metrics (macro-F1, balanced accuracy, per-class recall).

In [78]:

```
# Final class-imbalance check and concise EDA summary
import numpy as np
from IPython.display import Markdown, display

summary_dist = (
    df[target_name]
    .value_counts(dropna=False)
    .rename_axis("class")
    .reset_index(name="count")
)
summary_dist["pct"] = (summary_dist["count"] / len(df) * 100).round(2)

display(summary_dist)

largest_pct = float(summary_dist["pct"].max())
smallest_pct = float(summary_dist["pct"].min())
imbalance_ratio = np.nan if smallest_pct == 0 else round(largest_pct / smalles

print(f"Largest class percentage: {largest_pct:.2f}%")
print(f"Smallest class percentage: {smallest_pct:.2f}%")
```

```
print(f"Imbalance ratio (largest/smallest): {imbalance_ratio}")

if "missing_df" in globals() and len(missing_df) > 0:
    top_missing = missing_df.iloc[0]
    top_missing_text = f"{top_missing['feature']} ({top_missing['missing_pct']
else:
    top_missing_text = "No missing values detected"

if "duplicate_rows" in globals():
    dup_value = int(duplicate_rows)
elif "duplicate_count" in globals():
    dup_value = int(duplicate_count)
else:
    dup_value = int(df.duplicated().sum())

final_eda_conclusion = f"""
### Report-Ready EDA Summary

1. Duplicate row check found **{dup_value}** duplicate rows.
2. Highest missingness appears in **{top_missing_text}**, so imputation is rec
3. Target classes range from **{smallest_pct:.2f}%** to **{largest_pct:.2f}%**
4. Class imbalance ratio is **{imbalance_ratio}**; evaluate models with macro-
5. Correlation and distribution checks support moving to robust preprocessing
"""

display(Markdown(final_eda_conclusion))
```

	class	count	pct
0	Healthy	2121	36.57
1	Anemia	1805	31.12
2	Rickets_Osteomalacia	1429	24.64
3	Night_Blindness	269	4.64
4	Scurvy	176	3.03

```
Largest class percentage: 36.57%
Smallest class percentage: 3.03%
Imbalance ratio (largest/smallest): 12.07
```

Report-Ready EDA Summary

1. Duplicate row check found **0** duplicate rows.
2. Highest missingness appears in **alcohol_consumption (33.66%)**, so imputation is required in preparation.
3. Target classes range from **3.03%** to **36.57%**.
4. Class imbalance ratio is **12.07**; evaluate models with macro-F1, balanced accuracy, and per-class recall.
5. Correlation and distribution checks support moving to robust preprocessing and model comparison.

Created in **Deepnote**